

Task 5 Enhancement Report: Cold Start and Image Size

Ethan Bayarsaikhan (000961411) · CPSY 300 Project 1 · 2026-10-04 · Applied to Task 3 (serverless function) and Task 2 (container image)

1. Options chosen

- **Option 1, smaller and more efficient images:** a separate, function-only container image.
- **Option 2, cold start and execution speed:** reuse work across warm invocations of the function.

2. Research

- **Where cold-start time goes.** A cold invocation must fetch the image, start a container, start Python and import libraries, then do the work. A warm instance keeps its process alive, so module-level state survives between calls. I measured each stage before changing anything.
- **Smaller images.** Docker's build best practices advise against installing packages an image does not need. The function never draws charts, yet it shared an image with matplotlib and seaborn.
- **Reusing clients.** Microsoft's guidance on managing connections in Azure Functions recommends creating clients once and reusing them across invocations instead of per call.
- **ETags.** Azure Blob Storage gives every blob an ETag that changes whenever the blob is overwritten, so an unchanged ETag means unchanged content and the previous result is still valid.

3. Improvements applied

- **A: Function-only image** (`Dockerfile.function` , `requirements-function.txt`): only numpy, pandas and azure-storage-blob; only the three files the function imports; `pip --no-cache-dir`; modules precompiled with `compileall`; non-root user.
- **B: Warm path** (`lambda_function.py`): one Blob client per process, and a check of the CSV's ETag before downloading. If it matches the last processed version, the stored result is returned without downloading or recomputing. The ETag saved is the one from the download itself, so a CSV replaced mid-call is still picked up. Three new unit tests cover the cache hit, a changed CSV and a deleted results file.
- **Measurement:** `cold-start.yml` workflow with `cold_start_benchmark.py` . Each measure is the median of 5 runs after one discarded run, on a GitHub Actions runner against Azureite.

4. Results and impact

Median of 5 runs	Run #2 (before B): baseline → slim	Run #3 (after B): baseline → slim
Image size	405.8 → 322.1 MB (-21%)	405.8 → 322.2 MB (-21%)
Container start	0.204 → 0.210 s	0.216 → 0.160 s
Python start and imports	0.900 → 0.531 s	0.550 → 0.397 s
Cold invocation	1.156 → 0.783 s (-32%)	0.806 → 0.554 s (-31%)
Warm invocation (CSV unchanged)	0.036 → 0.036 s	0.002 → 0.002 s (-94% vs run #2)

- **A:** the image is 21% smaller, so less to download on every cold start and deployment, and cold invocations were 31–32% faster than the baseline image in both runs. Imports were also faster in the slim image; I did not isolate the cause.
- **B:** warm invocations dropped from 36 ms to about 2 ms in both images, because repeat calls make one metadata request instead of downloading and processing 7,806 rows. Cold invocations are unaffected by design: a new process starts with an empty cache.
- **Limits:** the same baseline image measured 1.156 s and 0.806 s in two runs, so shared CI runners are noisy; I compare images only within the same run. These are local simulations, not real Azure Functions, where image download time would make A matter more. Related: Docker layer caching in the CI pipeline cut the image build from 61 s to 9 s (runs #47 and #48).

Sources: Docker, "Building best practices" (docs.docker.com/build/building/best-practices) · Microsoft Learn, "Manage connections in Azure Functions" (learn.microsoft.com/azure/azure-functions/manage-connections) · Microsoft Learn, "Managing concurrency in Blob Storage" (learn.microsoft.com/azure/storage/blobs/concurrency-manage) · Python docs, "compileall" (docs.python.org/3/library/compileall.html) · Evidence: Cold start benchmark runs #2 and #3, [docs/evidence/task5-benchmark-before.png](#) and [-after.png](#) .